Assignment 3

Jason Wang(24839132) and Chalaka Nawarathna(24609438)

Today

Contents

1 Question 1 **2**

1.1 (a) . . . 2

1.2 (b) . . . 3

1.3 (c) . . . 5

2 Question 2 **8**

2.1 (a) . . . 8

2.2 (b) . . . 9

2.3 (c) . . . 10

2.4 (d) . . . 10

2.5 (e) . . . 12

3 Question 3 **13**

3.1 (a) . . . 13

3.2 (b) . . . 14

3.3 (c) . . . 14

4 Question 4 **15**

4.1 (a) . . . 15

4.2 (b) . . . 17

4.3 (c) . . . 17

Assignment 3 submitted by Jason Wang(24839132) and Chalaka Nawarathna(24609438)

This assignment was completed by Jason Wang(24839132) and Chalaka Nawarathna(24609438). Both Jason Wang(24839132) and Chalaka Nawarathna(24609438) have first individually done the assignment questions on their own and have met up to discuss and compare each question and answers and also both contributed to the explanation of each answer. Jason Wang(24839132) was responsible for the compilation of the final assignment and Chalaka Nawarathna(24609438) has contributed to the commenting, verifying and documentation of the codes.

```r
# loading in libraries used for the assignment
library(glmnet)
library(splines)
library(gam)
library(ISLR)
library(ggplot2)
library(patchwork)
```

1 Question 1

1.1 (a)

```r
# read dat.csv dataset into R and storing into dat variable
dat <- read.csv('dat.csv')
# splitting the x and y variables
x <- model.matrix(y~., dat)[,-1] # remove the column corresponding to intercept
y <- dat$y
# create grid of lambda values
lambdagrid <- exp(seq(-8, 8, length = 100))
```

We first split the variables for the x and y. Using model.matrix will automatically handle dummy encoding of categorical variables. The lambdagrid variable will span 100 values on a logscale, these will be used to cycle through different levels of regularization when fitting the ridge regression model, for each fold in the corss validation loop, the ridge model is fitted once but evaluated at every lambda value in the grid.

```r
# set seed so cross validation is reproducible
set.seed(1)
# number of folds
K <- 10
# number of training observations
n <- nrow(dat)
# randomly assign observations to 10 folds
folds <- sample(rep(1:K, length = n))
# vector to store cross validation errors
cv.errors <- rep(0,length(lambdagrid))

# perform 10 fold cross validation
for(k in 1:K){
  # indexes of observation for kth fold
  id <- which(folds==k)
  # fit training data into ridge regression
  ridge.mod.tmp <- glmnet(x[-id,],y[-id],alpha = 0, lambda = lambdagrid, thresh = 1e-12)
  # calculate the prediction error for each lambda
  for(lam in 1:length(lambdagrid)){
    # predicts on validation set
    pred.tmp <- predict(ridge.mod.tmp, s = lambdagrid[lam], newx = x[id,])
    # store and accumulate mean square error
    cv.errors[lam] <- cv.errors[lam] + length(id)/n*mean((pred.tmp-y[id])^2)
  }
}
```

We then partition the data randomly into 10 folds. For each fold, the model trains on the remaining 9 fold and validates on the held out fold. The weighted MSE accounts for fold size differences, giving a proper weighted average across fold. This is repeated for every lambda producing 100 CV error estimates. For ridge regression, a penalty is applied to the loss function. $RSS + \lambda \sum_{j=1}^{p} (\beta_j{}^2)$, where λ is the tuning parameter. The ridge regression seeks the coefficient estimate that minimizes the RSS, but the second term $\lambda \sum_{j=1}^{p} (\beta_j{}^2)$ (shrinkage penalty) is small when the β are close to zero. The tuning parameter λ controls the relative impact of these 2 terms. Applying 10 fold cv is necessary in order to determine the optimal λ that gives us the lowest MSE error.

```
# find index position of minimum cv error score
minimum_index <- which.min(cv.errors)
# pull the lambda with the smallest cv error score
best.lambda <- lambdagrid[minimum_index]
cat("The optimal lambda - Ridge:", round(best.lambda,4), "\n")
```

```
## The optimal lambda - Ridge: 0.0117
```

From the 10-fold CV, we can see that 0.0117 is the optimal lambda that minimizes the cv MSE. The lambda is relatively small suggesting that the data does not require heavy regularization and the predictors carry genuine signals and heavy regularization would hurt the model's performance. Moreover, the small lambda value suggests that the model behaves closely to an OLS, and that only a very small regularization is needed to the coefficients to prevent overfitting and while retaining the predictive power of the predictors.

```
# fit rigde regression using full dataset(alpha=0 to specify ridge)
l2.fit <- glmnet(x, y, alpha = 0, lambda = lambdagrid)
# calculate l2 norm of the coefficient with the best lambda value
l2.norm <- sqrt(sum(l2.fit$beta[,minimum_index]^2))
cat("The l2 norm of the optimal lambda - Ridge:", round(l2.norm,4), "\n")
```

```
## The l2 norm of the optimal lambda - Ridge: 0.8116
```

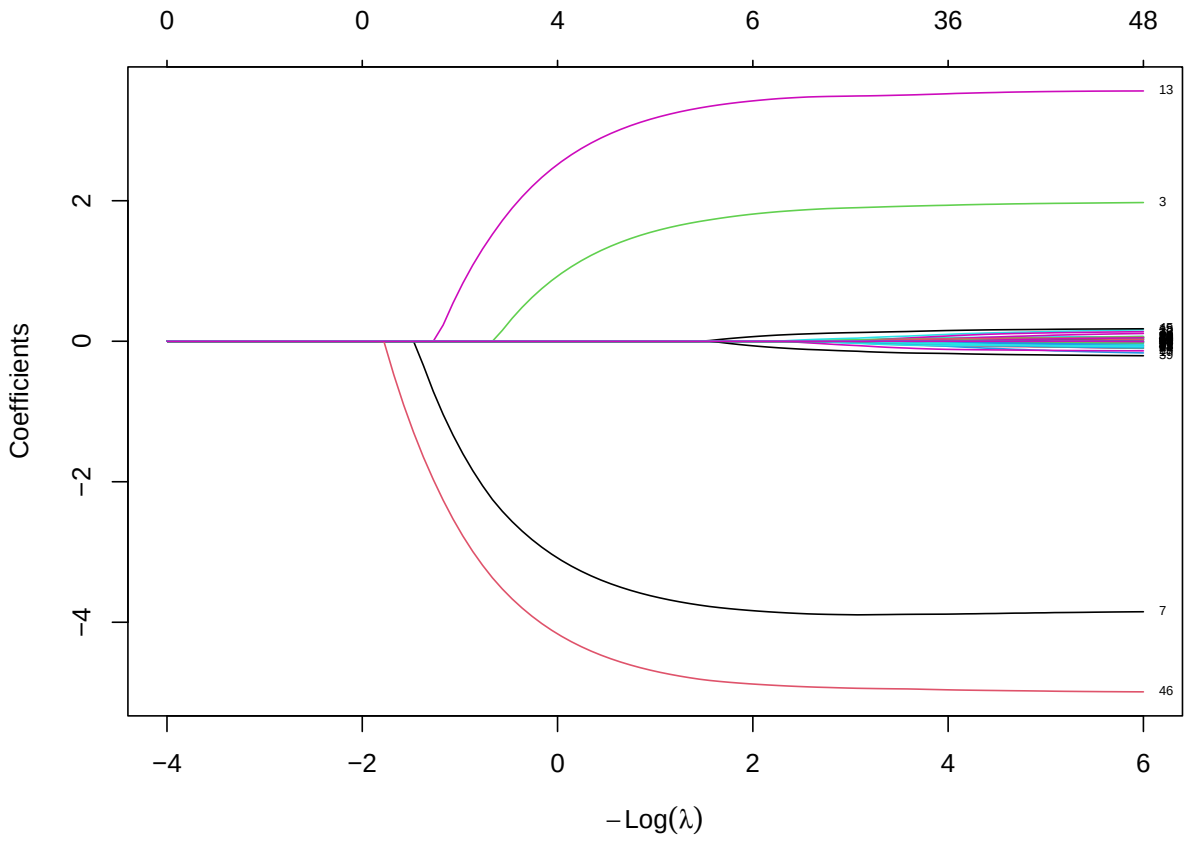
```
cat("The shrinkage penalty - Ridge:", round((best.lambda*(l2.norm^2)),4), "\n")
```

```
## The shrinkage penalty - Ridge: 0.0077
```

The l2-norm measures the overall magnitude of all coefficients combined showing how far the coefficient vector is from 0. At lambda = 0.0117, the l2-norm is 0.8116 indicating the coefficients retain most of their magnitude and have not been heavily penalized. This is consistent with the small optimal lambda, as the resulting shrinkage penalty of 0.0077 is very small, confirming that only mild shrinkage is applied. This overall supports that predictors carry genuine signals and do not reuire heavy penalization.

1.2 (b)

```
# create lambda grid for lasso regression
lambdagrid <- exp(seq(-6,4,length = 100))
# fit lasso regression using the training data
lasso.fit <- glmnet(x, y, alpha = 1, lambda = lambdagrid)
# plot coefficient paths against lambda values
plot(lasso.fit, xvar = 'lambda', label=TRUE)
```



The lasso trace plots display the behavior of the coefficient estimates as the tuning parameter λ varies. The x-axis shows moving to the left corresponds to increasing λ which results to increase penalty, moving to the left decreasing the λ which reduces the penalty. The y-axis represents coefficient estimates and the top represents number of predictors at each lambda value. Lastly, the lines represent the coefficients. With this, high values of lambda(y-axis), would lead to stronger penalty, which forces coefficients to move closer to zero(x-axis), which results in fewer predictors. While lower values of lambda leads to weaker penalty, resulting to more coefficients becoming non-zero and thus more predictors. From the plot, we can see that 4 predictors deviate the most from 0, suggesting these variables are stronger predictors compared to the rest. These values are 3, 7, 13, and 46. Overall, this plot shws the lasso regression variable selection property.

```
# display estimated coefficients when lambda = 2
coef(lasso.fit, s=2)
```

```
## 51 x 1 sparse Matrix of class "dgCMatrix"
##                    s=2
## (Intercept)  2.756025
## X1           .
## X2           .
## X3           .
## X4           .
## X5           .
## X6           .
## X7          -2.204093
## X8           .
```

```
## X9           .
## X10          .
## X11          .
## X12          .
## X13          1.470008
## X14          .
## X15          .
## X16          .
## X17          .
## X18          .
## X19          .
## X20          .
## X21          .
## X22          .
## X23          .
## X24          .
## X25          .
## X26          .
## X27          .
## X28          .
## X29          .
## X30          .
## X31          .
## X32          .
## X33          .
## X34          .
## X35          .
## X36          .
## X37          .
## X38          .
## X39          .
## X40          .
## X41          .
## X42          .
## X43          .
## X44          .
## X45          .
## X46         -3.323124
## X47          .
## X48          .
## X49          .
## X50          .
```

If we set lambda equals to 2, the predictors with non zero values are x7, x13 and x46, suggesting that these 3 predictors carry the most weight in the data. The lasso penalty has shrunk all remaining predictors to exactly 0, thus performing an automatic variable selection process and identifies x7, x13 and x46 as the onl meaningful contributors to the model at this specific level of regularization.

1.3 (c)

```
# lambda grids for both ridge and lasso regression
ridge.grid <- exp(seq(-8,8,length = 100))
```

```r
lasso.grid <- exp(seq(-6,4,length = 100))
# vectors to store cross validation errors
ridge.cv.errors <- rep(0,length(ridge.grid))
lasso.cv.errors <- rep(0,length(lasso.grid))
# number of folds
K <- 10
# number of training observations
n <- nrow(dat)
# set seed for reproducibility
set.seed(1)
# randomly assign observations to 10 folds
folds <- sample(rep(1:K, length = n))
# perform 10 fold cv for ridge regression
for(k in 1:K){
  # observation indexes in validation fold
  id <- which(folds==k)
  # perform inner cross validation to choose best lambda from ridge regression
  ridge.cv <- cv.glmnet(x[-id,],y[-id],alpha = 0, lambda = ridge.grid, thresh = 1e-12)
  # store best lambda from inner cv above
  best.lambda <- ridge.cv$lambda.min
  # predict on validation set
  ridge.pred <- predict(ridge.cv, s = best.lambda, newx = x[id,])
  # store mean square prediction error of ridge regression
  ridge.cv.errors[k] <- mean((y[id]-ridge.pred)^2)
}

for(k in 1:K){
  # observation indexes in validation fold
  id <- which(folds==k)
  # perform inner cross validation to choose best lambda from lasso regression
  lasso.cv <- cv.glmnet(x[-id,],y[-id],alpha = 1, lambda = lasso.grid, thresh = 1e-12)
  # store best lambda from inner cv
  best.lambda <- lasso.cv$lambda.min
  # predict on validation set
  lasso.pred <- predict(lasso.cv, s = best.lambda, newx = x[id,])
  # store mean square prediction error of lasso regression
  lasso.cv.errors[k] <- mean((y[id]-lasso.pred)^2)
}

# calculate the average cross validation error for both the ridge and lasso regression

cat("CV MSE - Ridge:", round(mean(ridge.cv.errors), 4), "\n")
```

```
## CV MSE - Ridge: 0.0634
```

```r
cat("CV MSE - Lasso:", round(mean(lasso.cv.errors), 4), "\n")
```

```
## CV MSE - Lasso: 0.0294
```

To compare the ridge and lasso regression, nested 10 fold cv was performed. The outer loop partitions the data into 10 folds, for each fold, the model trains on the remaining 9 fold and validates it on the held out fold. Within each outer fold, an inner cv is performed on the partitioned data to select the optimal lambda

from the respective lambda grids. The outer fold prediction error is then computed using the optimal λ selected from the inner CV and the process is repeated across all 10 folds. Then, the final MSE is averaged for all 10 outer folds.

From the results, we see that lasso regression achieved an overall lower MSE score compared to the ridge model. Thus, lasso is the better approach for this dataset. This is possible due to the earlier observation that only a small number of predictors have higher weights, where lasso's ability to shrink irrelevant variables to 0 gives it a clear advantage over ridge, which retains all predictor model, resulting in higher MSE score.

With that we run the lasso regression to find the best lambda and best coefficients. We then see that coefficients x3, x7, x12, x13, x15, x16, x17, x18, x25, x35, x37, x39, x45, x46.

```
# set seed for reproducibility
set.seed(1)
# full model for the lasso regression
final.cv <- cv.glmnet(x, y, alpha = 1, lambda = lasso.grid, nfolds = 10)
# best lambda that minimizes the error
best.lambda.lasso <- final.cv$lambda.min
# final lasso model with the lambda set with the best lambda value
final.lasso <- glmnet(x, y, alpha = 1, lambda = best.lambda.lasso)
# pull the coefficients from the best lambda
final_coefs <- coef(final.lasso)
print(final_coefs)
```

```
## 51 x 1 sparse Matrix of class "dgCMatrix"
##                        s0
## (Intercept)  3.207836172
## X1           .
## X2           .
## X3           1.892632609
## X4           .
## X5           .
## X6           .
## X7          -3.890501839
## X8           .
## X9           .
## X10          .
## X11          .
## X12          0.013149393
## X13          3.486322750
## X14          .
## X15         -0.007739993
## X16          0.012111912
## X17         -0.016314685
## X18          0.036612304
## X19          .
## X20          .
## X21          .
## X22          .
## X23          .
## X24          .
## X25         -0.047975308
## X26          .
## X27          .
## X28          .
```

```
## X29          .
## X30          .
## X31          .
## X32          .
## X33          .
## X34          .
## X35          0.011984028
## X36          .
## X37         -0.017495916
## X38          .
## X39         -0.133401024
## X40          .
## X41          .
## X42          .
## X43          .
## X44          .
## X45          0.118151998
## X46         -4.932846136
## X47          .
## X48          .
## X49          .
## X50          .
```

We then write the model as:

$$\hat{y} = 3.2078 + 1.8926x_3 - 3.8905x_7 + 0.0131x_{12} + 3.4863x_{13} + 0.0077x_{15} + 0.0121x_{16} - 0.0163x_{17} + 0.0366x_{18} - 0.0480x_{25} + 0.0120x_{35} - 0.0175x_{37} - 0.1334x_{39} + 0.1182x_{45} - 4.9328x_{46}$$

2 Question 2

2.1 (a)

```
# read the data file
q2 <- read.csv('q2.csv')
# check data types
str(q2)
```

```
## 'data.frame':    234 obs. of  2 variables:
##  $ Y: int  1 0 0 1 0 0 0 0 1 0 ...
##  $ X: num  -1.21 -0.82 -0.653 0.499 -1.933 ...
```

The Y variable in the q2 dataset is set as int, the variable is actually binary of 0 and 1. This will be factoriezed.

```
#Factorizing the Y variable
q2$Y <- as.factor(q2$Y)
# check data types
str(q2)
```

```
## 'data.frame':    234 obs. of  2 variables:
##  $ Y: Factor w/ 2 levels "0","1": 2 1 1 2 1 1 1 1 2 1 ...
##  $ X: num  -1.21 -0.82 -0.653 0.499 -1.933 ...
```

```
# fit logistic regression model
log.fit <- glm(Y~X, data = q2, family = binomial)
# output the new probability
log.proba <- predict(log.fit, newdata = q2, type = 'response')
# converts the probabilities into class labels
log.pred <- ifelse(log.proba > 0.5, 1, 0)
# calculate and print out the classification error
err_a <- mean(log.pred != q2$Y)
cat("Classification error:", round(err_a, 4), "\n")
```

```
## Classification error: 0.3974
```

A logistic regression model was fitted to predict Y based on X, modeling the probability of class memberships as:

$$P(Y = 1|X) = \frac{e^{\beta_0+\beta_1 x}}{1 + e^{\beta_0+\beta_1 x}}$$

The logistic regression model achieved a classification error of 0.3974, meaning it misclassifies 39.74% of the obervations. This is relatively high, suggesting that a simple linear decision boundary is not enough to capture the relationship betwee Y and X.

2.2 (b)

```
# fit a polynomial model with degree 3
poly.fit <- glm(Y ~ poly(X, 3, raw = TRUE), data = q2, family = binomial)
# output the class probabilities
poly.proba <- predict(poly.fit, newdata = q2, type = 'response')
# convert the probabilities into class labels
poly.pred <- ifelse(poly.proba > 0.5, 1, 0)
# calculate and print out the classification error
err_b <- mean(poly.pred != q2$Y)
cat("Classification error:", round(err_b, 4), "\n")
```

```
## Classification error: 0.2094
```

```
# check model coefficients
coef(poly.fit)
```

```
##             (Intercept) poly(X, 3, raw = TRUE)1 poly(X, 3, raw = TRUE)2
##            -0.009656213             1.918952608            -0.523486704
## poly(X, 3, raw = TRUE)3
##            -1.402647807
```

A polynomial logistic regression model of degree 3 was fitted to predict Y based on X, modeling the probability of class memberships as:

$$P(Y = 1|X) = \frac{e^{\beta_0+\beta_1 x+\beta_2 x^2+\beta_3 x^3}}{1 + e^{\beta_0+\beta_1 x+\beta_2 x^2+\beta_3 x^3}}$$

The polynomial logistic regression model achieved a classification error of 0.2094, meaning it misclassifies 20.94% of the observations. Compared to the logistic model, it has a lower classification error. This suggests that the relationship between X and Y is non-linear and that the cubic decision boudary captures the overll structure of the data considerably way better.

The full model is written as:

$$P(Y = 1|X) = \frac{e^{-0.0097+1.9190x-0.5235x^2-1.4026x^3}}{1 + e^{-0.0097+1.9190x-0.5235x^2-1.4026x^3}}$$

2.3 (c)

```r
# fir the natural cubic spline into a logistic regression model
spline.fit <- glm(Y ~ ns(X, knots = c(-0.5, 0.5, 1)), data = q2, family = binomial)
# output the class probabilities
spline.proba <- predict(spline.fit, newdata = q2, type = 'response')
# convert probabilities into class labels
spline.pred <- ifelse(spline.proba > 0.5, 1, 0)
# calculate and print out the classification error
err_c <- mean(spline.pred != q2$Y)
cat("Classification error:", round(err_c, 4), "\n")
```

```
## Classification error: 0.2051
```

```r
# find the number of columns of the basis matrix
basis <- ns(q2$X, knots = c(-0.5, 0.5, 1))
cat("Columns in basis matrix:", ncol(basis), "\n")
```

```
## Columns in basis matrix: 4
```

A logistic regression with natural cubic splines is fitted using 3 knots at -0.5, 0.5 and 1. The basis matrix contains 4 columns. This means instead of fitting a single straight line to separate the classes, the natural cubic spline divides the range of X into segments at knots -0.5, 0.5 and 1, and fits a smooth piecewise cubic curve in each segment. Each of the 4 columns represent one of these curved piece, providing flexibility for the model to adapt to the non-linearity of the relationship between X and Y. The natural spline further constrain the the curve to be linear beyond the boundary knots, preventing over fitting at the extremes. This is reflected by the lowest classification(0.2051) it achieved among the 3 models.

2.4 (d)

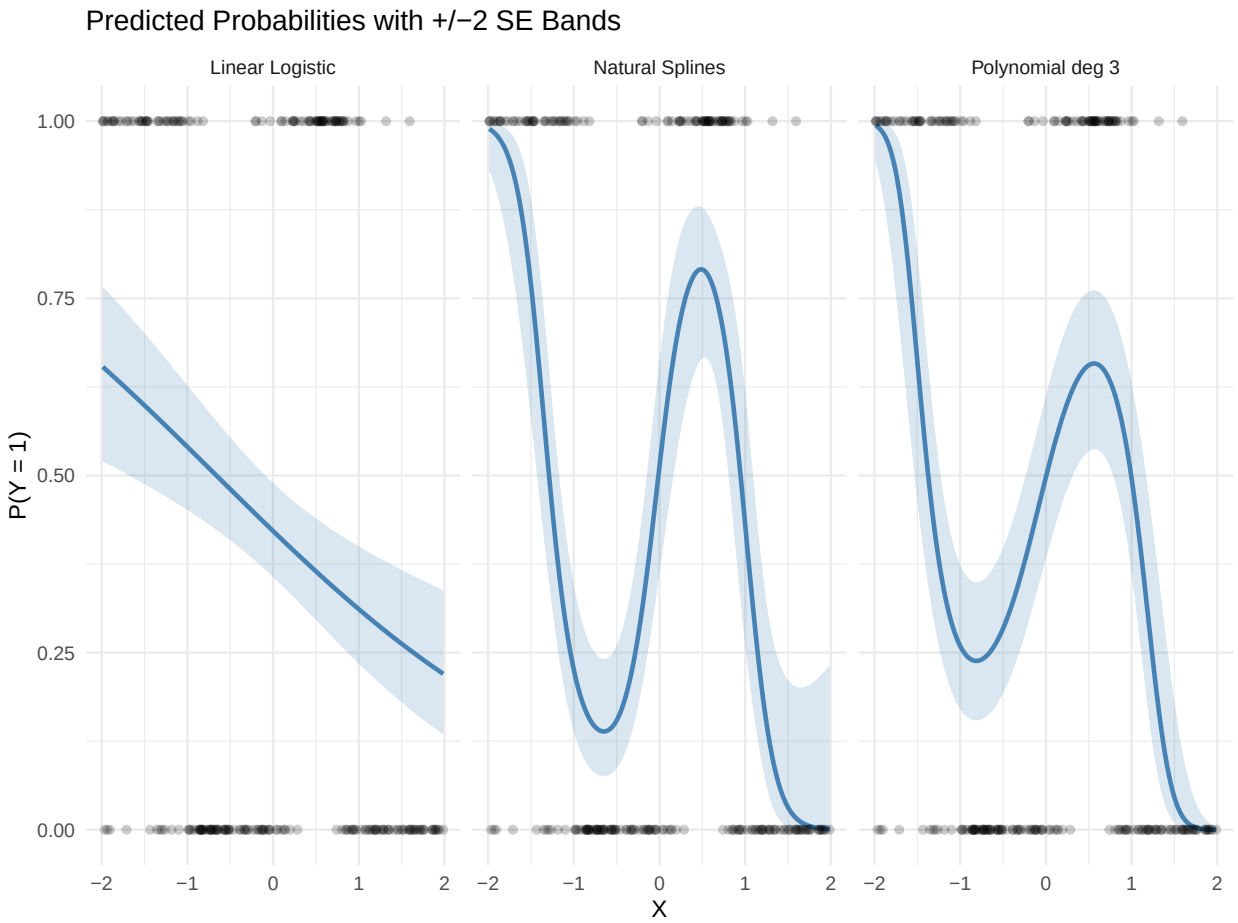
```r
# Plotting with +/- 2 SE bands
# Create a fine grid of X values
x_seq <- seq(min(q2$X), max(q2$X), length = 300)
new_data <- data.frame(X = x_seq)

# Get predictions on the link scale (log-odds), then convert to probability
get_pred_df <- function(fit, new_data, model_name) {
  pred <- predict(fit, newdata = new_data, type = "link", se.fit = TRUE)
  data.frame(
```

```
    x     = new_data$X,
    prob  = plogis(pred$fit),                        # convert to probability
    lo    = plogis(pred$fit - 2 * pred$se.fit),      # lower band
    hi    = plogis(pred$fit + 2 * pred$se.fit),      # upper band
    model = model_name
  )
}

df_all <- rbind(
  get_pred_df(log.fit, new_data, "Linear Logistic"),
  get_pred_df(poly.fit, new_data, "Polynomial deg 3"),
  get_pred_df(spline.fit, new_data, "Natural Splines")
)

ggplot(df_all, aes(x = x, y = prob)) +
  geom_line(color = "steelblue", linewidth = 1) +
  geom_ribbon(aes(ymin = lo, ymax = hi), alpha = 0.2, fill = "steelblue") +
  geom_point(data = q2, aes(x = X, y = as.numeric(as.character(Y))), alpha = 0.2, inherit.aes = FALSE) +
  facet_wrap(~model) +
  labs(title = "Predicted Probabilities with +/-2 SE Bands",
       x = "X", y = "P(Y = 1)") +
  theme_minimal()
```



The plot displays the predicted probabilities with 2*SE bands for all 3 models across the range of X.

Linear Logistic Regression: The plot shows the predicted probability decreasing monotonically from left to right, reflecting the rigid linear decision boundary. The somewhat narrow SE bands in the middle portion shows somewhat high confidence, though the farther the points are from the center SE band shows slightly lower confidence. However, the model fails to capture the non-linearity of the dataset, resulting in a very high classification error of 0.3974.

Natural Splines: The predicted probability oscillates multiple times across the ranges of X, reflecting a highly flexible and non linear decision boundary. The SE bands show that is has high confidence in some regions and lower confidence in others, suggesting some uncertainty in its predictions. However the model did achieve the lowest classification error of 0.2051.

Polynomial degree 3 Logistic Regression: The plot shows the predicted probability shows a smoother non-linear curve, reflecting it's flexibility in capturing the non-linearity of the data. We can also see the SE bands are very narrow, indicating high confidence in its predictions. Compared to the spline, the polynomial curve is smoother and less wiggly, though it is less flexible in adapting to local patterns in the data.The model did reasonably well in achieving a classification error of 0.2094, though not as well as the spline model.

Overall, the polynomial and spline outperform the linear logsitic model, confirming that X and Y are non linear. Though, the natural spline is the better model out of the 2.

2.5 (e)

```
# set seed for reproducibility
set.seed(4064)
n = 234
K = 10
idx = sample(n,n)
nk = c(rep(23,9),n-23*9)
idx1 = cumsum(c(0,nk))
folds = list()

for(k in 1:K){
  folds[[k]] = idx[(idx1[k]+1):idx1[k+1]]
}

log.cv <- rep(0, K)
poly.cv <- rep(0, K)
spline.cv <- rep(0, K)

for(k in 1:K){
  id <- folds[[k]]
  log.fit.tmp <- glm(Y~X, data = q2[-id,], family = binomial)
  log.proba <- predict(log.fit.tmp, newdata = q2[id,], type = 'response')
  log.pred <- ifelse(log.proba > 0.5, 1, 0)
  log.cv[k] <- mean(log.pred != q2$Y[id])

  poly.fit.tmp <- glm(Y~poly(X, 3, raw = TRUE), data = q2[-id,], family = binomial)
  poly.proba <- predict(poly.fit.tmp, newdata = q2[id,], type = 'response')
  poly.pred <- ifelse(poly.proba > 0.5, 1, 0)
  poly.cv[k] <- mean(poly.pred != q2$Y[id])

  train <- q2[-id,]
  test <- q2[id,]
  ns.basis <- ns(train$X, knots = c(-0.5, 0.5, 1))
```

```
  train.ns <- data.frame(ns.basis, Y=train$Y)
  test.ns <- data.frame(predict(ns.basis, newx = test$X))
  colnames(test.ns) <- colnames(train.ns)[-ncol(train.ns)]
  spline.fit.tmp <- glm(train$Y~., data=train.ns, family = binomial)
  spline.proba <- predict(spline.fit.tmp, newdata = test.ns, type = 'response')
  spline.pred <- ifelse(spline.proba > 0.5, 1, 0)
  spline.cv[k] <- mean(spline.pred != test$Y)
}
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
mean(log.cv)
```

```
## [1] 0.4066023
```

```
mean(poly.cv)
```

```
## [1] 0.2165862
```

```
mean(spline.cv)
```

```
## [1] 0.1887279
```

A 10 fold cv was implemented to compare the 3 models, we partition the observations into 10 folds. For each fold, the model is trained on the remaining 9 folds and validated on the held-out fold, with the average classification error reported across all 10 folds.

From the results, we can see that the natural cubic spline model achieves the lowest cv classification error of 0.1887, making it the best performing model among the 3. This is consistent with the classification error as observed in part a, b and c, confirming that the spline model generalizes the best. The linear logistic model performs the worst with a CV error of 0.4066, confirming that the linear decision boundary is not enough to capture the non-linear relationship. Lastly, the polynomial degree 3 model has a lower cv error (0.2166) compared to the linear logistic model, but the natural cubic spline's ability to fit a flexible piecewise boundary while also able to constrain the curve gives a slightly better advantage, reducing the cv error more compared to the polynomial model. Overall, this suggests that the natural spline t be the best model to capture the non linearity of the X and Y.

3 Question 3

3.1 (a)

```
# Converting origin to Factor
Auto$origin <- factor(Auto$origin)
# set seed for reproducibility
set.seed(2345)
# splitting the dataset into train and test split
train.id <- sample(nrow(Auto),nrow(Auto)/2)
train.Auto <- Auto[train.id,]
```

```r
test.Auto <- Auto[-train.id,]
# fit the GAM model 1
gam1 <- gam(mpg ~ s(displacement, df=5) + s(acceleration, df=5) + s(horsepower, df=4) + s(weight, df=4)
# calculate the output predictions on the test set
pred1 <- predict(gam1, newdata = test.Auto)
# calculate and output the MSE
print(paste('MSE of model 1 is: ', round(mean((pred1-test.Auto$mpg)^2), 4)))
```

```
## [1] "MSE of model 1 is:  13.7713"
```

We created Model 1 1using splines for predictors (displacement and accelartion (df =5) and horsepower and weight (df=4)), and our model 1 got validation MSE of 13.7713. It tells us how well the model predicts mpg on unseen data.

3.2 (b)

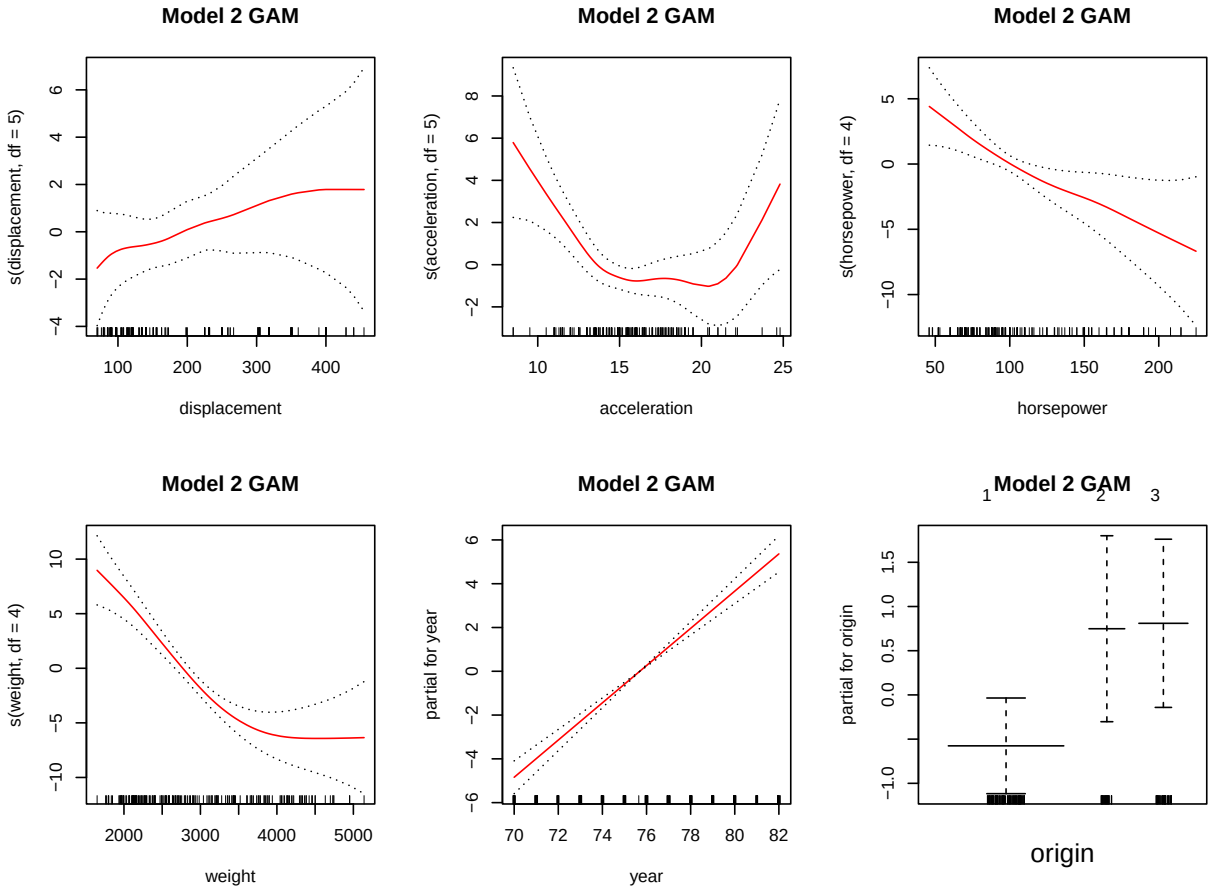
```r
# fit the GAM model 2
gam2 <- gam(mpg ~ s(displacement, df=5) + s(acceleration, df=5) + s(horsepower, df=4) + s(weight, df=4)
# calculate the output predictions on the test set
pred2 <- predict(gam2, newdata = test.Auto)
# calculate and output the MSE
print(paste('MSE of model 2 is: ', round(mean((pred2-test.Auto$mpg)^2), 4)))
```

```
## [1] "MSE of model 2 is:  8.2724"
```

We created Model 2 by updating Model 1 by adding the year and the factorized origin as linear predictors. The validation MSE for Model 2 was 8.2724. Since this is lower than the validation MSE for Model 1(13.7713), Model 2 performs better on the validation set and this suggest that the year and origin has an impact on predicting mpg.

3.3 (c)

```r
#plot the result of Model 2
par(mfrow = c(2, 3))
plot(gam2, se = TRUE, col = "red", main = "Model 2 GAM")
```



The GAM plots show the partial effect of each predictor on mpg with +/- 2SE bands. Displacements shows a slight positive curve while acceleration shows a U shape curve and both have wide confidence bands indicating more uncertainty.Horsepower and weight have strong negative non linear effects as they increase, mpg decreases. Year has a strong positive linear effect, with newer cars being more fuel efficient. When we look at the origin we can see that origin categories 2 and 3 (European and Japanese) cars tend to have a higher mpg compared to origin category 1 (American) cars.

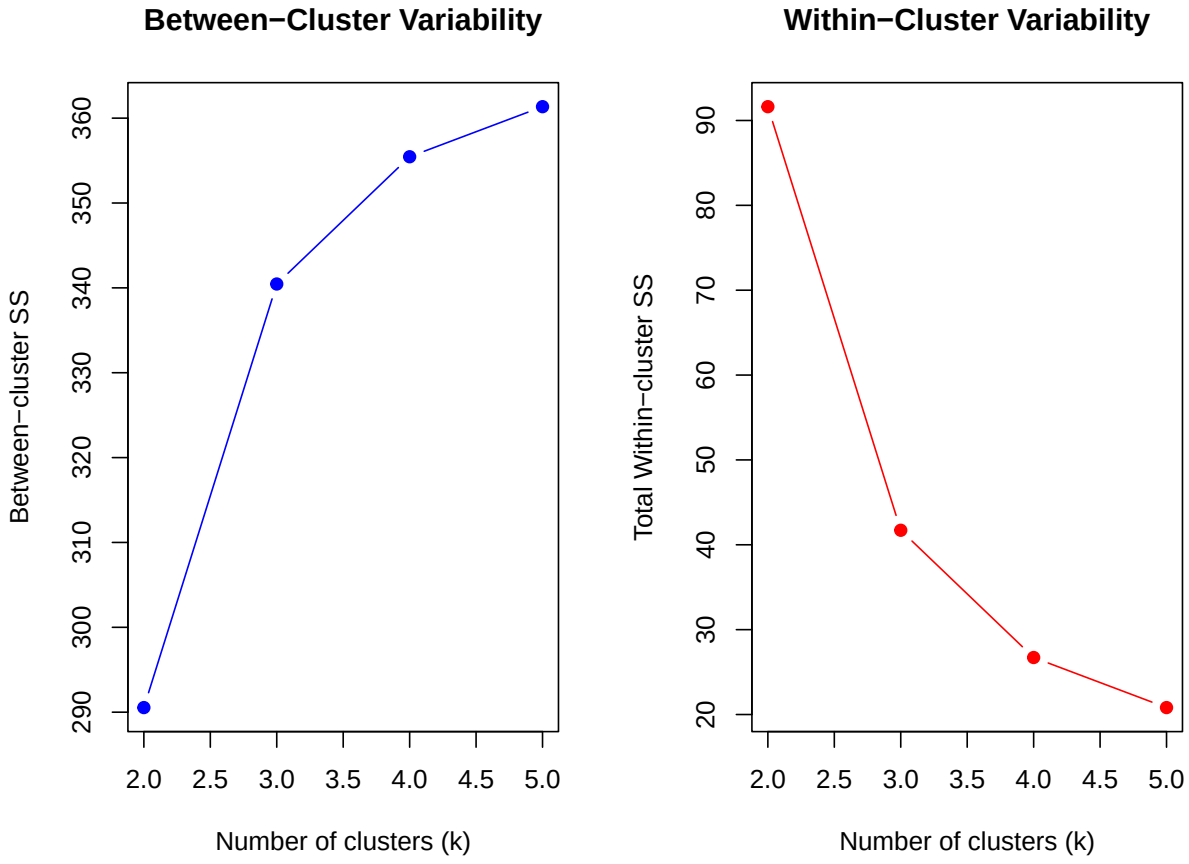
4 Question 4

4.1 (a)

```
# read in the retail dataset
retail<- read.csv('retail.csv')
# set seed for reproducibility
set.seed(1)
# set the K from 2 to 5
K <- c(2, 3, 4, 5)
# vectors to store the betweeness and withiness clusters
tot.within <- rep(0, length(K))
between.within <- rep(0, length(K))
# for loop to loop through each k cluster and save the betweeness and withiness clusters into the vecto
```

```r
for(i in 1:length(K)){
  k <- K[i]
  k_means <- kmeans(retail, centers = k, nstart=20)
  tot.within[i] <- k_means$tot.withinss
  between.within[i] <- k_means$betweenss
}

par(mfrow = c(1, 2))
# plot the number of clusters against the between cluster variability
plot(K, between.within, type = "b", pch = 19, col = "blue",
     xlab = "Number of clusters (k)",
     ylab = "Between-cluster SS",
     main = "Between-Cluster Variability")
# plot the number of clusters against the within cluster variability
plot(K, tot.within, type = "b", pch = 19, col = "red",
     xlab = "Number of clusters (k)",
     ylab = "Total Within-cluster SS",
     main = "Within-Cluster Variability")
```



When we look at the between cluster variability plot it shows that when number of clusters (k) increases between cluster variability increases too. There is a large jump of cluster SS from around 305 at k =2 to 365 at k=3. After that increase becomes more gradual where it gets small gains at k =3,k=4,k=5. On the other hand when k increases total within cluster variability decreases. There is a steep drop when k =2 (105) to k

=3 (50), and after that decreases becomes more gradual. This suggest that an elbow at k = 3, which means adding more than 3 clusters does not substantially reduce within cluster variability. k= 3 seems to be the ideal choice.

4.2 (b)

```
# scale the retail dataset
retail.scaled <- as.data.frame(scale(retail))
# set seed for reproducibility
set.seed(1)
# calculate the kmeans of the scaled dataset with k=3
k_means_3 <- kmeans(retail.scaled, 3, nstart=20)
retail.scaled$cluster <- as.factor(k_means_3$cluster)
# How many customers in each cluster..
cat("Cluster sizes:", k_means_3$size, "\n")
```

```
## Cluster sizes: 5 10 15
```

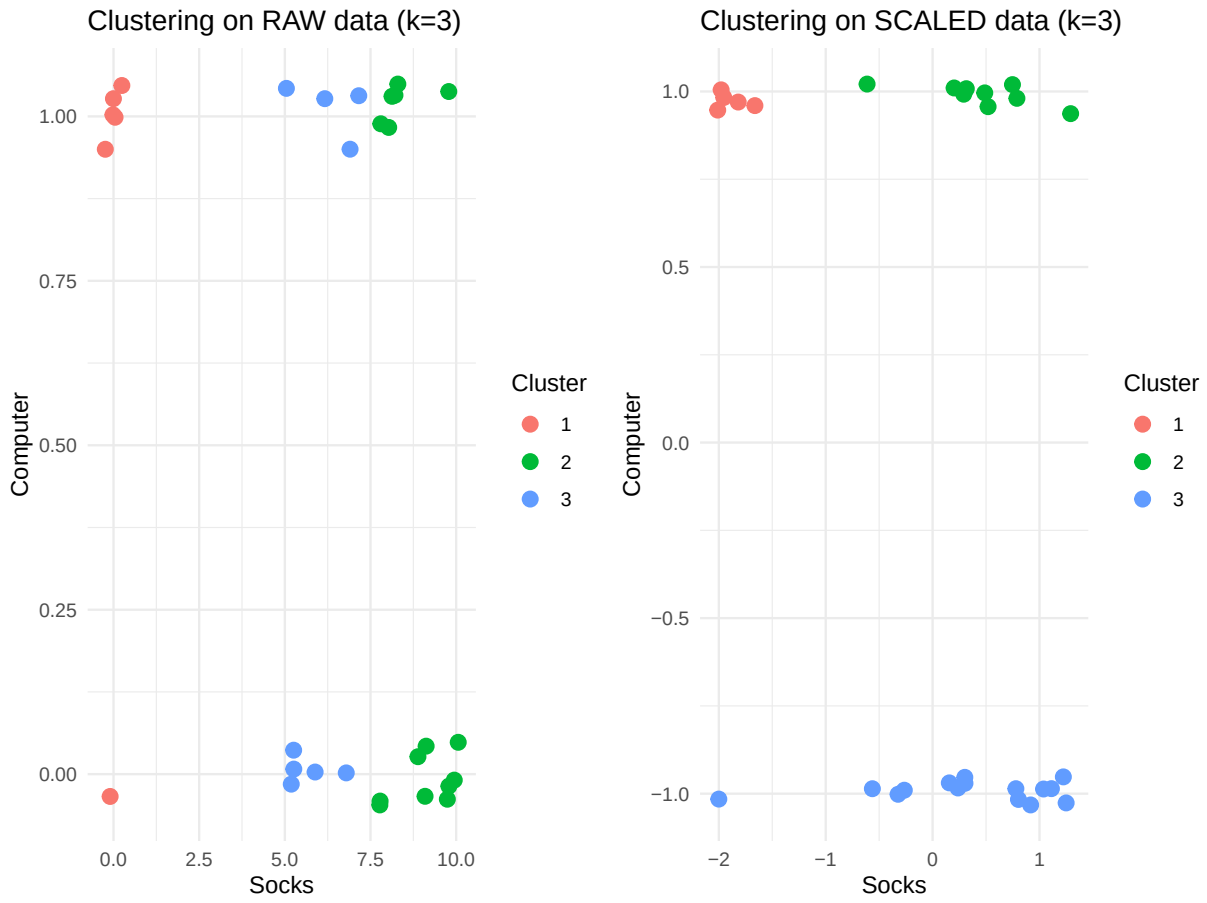
```
table(retail.scaled$cluster)
```

```
##
##  1  2  3
##  5 10 15
```

After scaling the retail data to have mean 0 and variance 1, k means clustering with 3 produce clusters sizes of 5, 10, and 15. Scaling ensure that all three variables (socks, phone and computer) contribute equally to the clustering, without considering their measurement scales.

4.3 (c)

```
#set seed for reproducibility
set.seed(1)
# calculate the kmeans of unscaled dataset with k=3
kmeans_original <- kmeans(retail, 3, nstart=20)
retail$cluster <- as.factor(kmeans_original$cluster)
# Plot 1: raw data clustering
p1 <- ggplot(retail, aes(x = socks, y = computer, color = cluster)) +
  geom_jitter(width = 0.3, height = 0.05, size = 3) +
  labs(title = "Clustering on RAW data (k=3)",
       x = "Socks", y = "Computer", color = "Cluster") +
  theme_minimal()
# Plot 2: scaled data clustering
p2 <- ggplot(retail.scaled, aes(x = socks, y = computer, color = cluster)) +
  geom_jitter(width = 0.3, height = 0.05, size = 3) +
  labs(title = "Clustering on SCALED data (k=3)",
       x = "Socks", y = "Computer", color = "Cluster") +
  theme_minimal()
# print plots side by side
p1+p2
```



The two scatter plots shows that the data naturally separated by customers who bought a computer (top band, computer = 1) and the customers who didnt (bottom band, computer = 0), while socks varying along the horizontal axis.

When we look at the twp plots we can see that the cluster assignments are not identical. In the raw data cluster it seems to be dominated by socks(0-10) because it had a large numerical range where computer(0-1) did not. So, this means that the computer purchases have a little influence in groupings of raw data.

In the plot with scaled data where all three variables contribute equally to the distance calculations. Because of that cluster assignments are different in this plot. Especially for customers who bought socks in the middle range, for them computer variables has a big impact on deciding their group.

So, by analyzing both plots we can come to the conclusion that its better to scale the data if not k means are just getting clustered based on socks and other variables wont be considered that much. But when we scale its make sure its fair and the clustering will much more meaninful cause every variable will be considered equally.